



MODUL 10 • VALIDASI DATA & FORMS

Mengapa Runtime Validation Penting



TypeScript hanya bekerja saat compile. Data dari user selalu bisa berbahaya di runtime.

TypeScript ≠ Runtime Validation

TypeScript tipe-check kode kita — bukan data dari luar. Data user, API, database bisa berbeda.

✗ TypeScript saja tidak cukup

✓ TypeScript + Zod = aman

```
// TypeScript compile-time: OK ✓  
interface CreatePostInput {  
  title: string  
  content: string  
}  
  
async function createPost(data: CreatePostInput)  
{ // TypeScript PERCAYA data.title adalah string  
  // tapi saat runtime — kita tidak tahu!  
  await db.post.create({ data })  
}  
  
// Yang bisa terjadi di runtime:  
// { title: "", content: null }  
// { title: "<script>xss</script>", ... }  
// { title: "A".repeat(100000), ... }  
// TypeScript tidak menangkap ini!  
// Bug ditemukan OLEH USER di production.
```

```
import { z } from 'zod'  
  
const createPostSchema = z.object({  
  title: z.  
    .string()  
    .min(3, 'Minimal 3 karakter')  
    .max(100, 'Maksimal 100 karakter')  
    .trim(),  
  content: z.string().min(10, 'Konten terlalu pendek'),  
})  
  
type CreatePostInput = z.infer<typeof createPostSchema>  
  
async function createPost(rawData: unknown) {  
  const result = createPostSchema.safeParse(rawData)  
  if (!result.success) {  
    return { errors: result.error.flatten().fieldErrors }  
  }  
  // result.data sudah typed dan validated!  
  await db.post.create({ data: result.data })  
}
```

Zod Basics — Core Concepts

bun add zod. Satu schema = runtime validation + TypeScript type.

 Zod fundamentals

```
import { z } from "zod"

// — Primitives —————
const nameSchema = z.string().min(2).max(50).trim()
const emailSchema = z.string().email("Email tidak valid")
const ageSchema = z.number().int().min(0).max(150)
const slugSchema = z.string().regex(/^[a-z0-9-]+$/, "Slug tidak valid")

// — safeParse: TIDAK throw — selalu gunakan ini di Server Action —————
const result = emailSchema.safeParse("bukan-email")
if (!result.success) {
  console.log(result.error.issues[0].message) // "Email tidak valid"
}
// result.success=true → result.data tersedia (typed!)
// result.success=false → result.error.flatten().fieldErrors tersedia

// — z.object — untuk form schema —————
const postSchema = z.object({
  title: z.string().min(3, "Min 3 karakter").max(100),
  content: z.string().min(10, "Konten terlalu pendek"),
  published: z.boolean().default(false),
})

// — z.infer — derive TypeScript type dari schema —————
type Post = z.infer<typeof postSchema>
// type Post = { title: string; content: string; published: boolean }
// Tidak perlu definisi type manual!
```

z.object, Optional, Default & refine

Compose schema kompleks. Validasi lintas-field dengan refine().

z.object — advanced patterns

```
// — Optional, default, nullable —————  
const schema = z.object({  
  title:      z.string().min(3),  
  excerpt:    z.string().max(200).optional(), // boleh undefined  
  coverImage: z.string().url().nullable(),    // boleh null  
  published:  z.boolean().default(false),      // default jika tidak ada  
  tags:       z.array(z.string()).max(5, "Maks 5 tag"),  
  categoryId: z.string().uuid("Kategori tidak valid"),  
})  
type CreatePost = z.infer<typeof schema>  
// {title:string; excerpt?:string; coverImage:string|null; published:boolean; ...}  
  
// — refine — validasi lintas-field —————  
const passwordSchema = z.object({  
  password:      z.string().min(8, "Min 8 karakter"),  
  confirmPassword: z.string(),  
}).refine(  
  data => data.password === data.confirmPassword,  
  { message:"Password tidak cocok", path:["confirmPassword"] }  
)  
  
// — Transform — ubah data setelah validasi —————  
const emailSchema = z.string().email().toLowerCase().trim()  
// Masukan "USER@EXAMPLE.COM " → output "user@example.com"
```

Evaluasi Pemahaman

Jawab sebelum lanjut.

Q1 — Mengapa TypeScript saja tidak cukup untuk validasi data dari user?

A) TypeScript hanya bekerja saat compile — data runtime tidak dicek

B) TypeScript tidak mendukung string

C) TypeScript terlalu lambat

Q2 — Apa perbedaan safeParse dan parse?

A) Tidak ada bedanya

B) safeParse return {success, data/error} — tidak throw. parse throw ZodError jika gagal.

C) safeParse lebih strict

Q3 — Apa fungsi z.infer<typeof schema>?

A) Menjalankan validasi

B) Export schema ke JSON

C) Derive TypeScript type dari schema — satu source of truth, tidak perlu duplikasi

Tugas Mandiri

Kenali Zod dan buat schema untuk project blog.

01 Install & Schema

bun add zod. Buat lib/validations/post.ts. Definisikan createPostSchema: title (min 3, max 100, trim), content (min 10), published (boolean, default false).

02 `z.infer` Types

Export type CreatePostInput = `z.infer<typeof createPostSchema>`. Gunakan type ini di semua fungsi yang handle post. Hapus manual interface jika ada.

03 Test `safeParse`

Buat test: parse data valid → data tersedia dan typed. Parse invalid → `error.flatten().fieldErrors` berisi pesan error yang tepat per field.

04 `refine` password

Buat passwordSchema dengan `refine`: `password === confirmPassword`. Jika berbeda, error di path 'confirmPassword'.

💡 `z.infer` = single source of truth. Satu schema → satu type. Tidak perlu duplikasi interface dan schema.

Yang Sudah Kita Pelajari

- ✓ TypeScript hanya compile-time. Data runtime dari user harus divalidasi secara eksplisit.
- ✓ Zod: runtime validation + TypeScript type generation. Dua manfaat dari satu schema.
- ✓ `safeParse`: return `{success, data/error}` — tidak throw. SELALU gunakan ini di Server Action.
- ✓ `z.object` untuk form schema. `optional()`, `default()`, `nullable()`, `refine()` untuk kasus khusus.
- ✓ `z.infer<typeof schema>`: derive TypeScript type — satu schema, satu type, tidak ada duplikasi.

→ Selanjutnya: **Bab 2 — Zod Schema & Inference Advanced**

